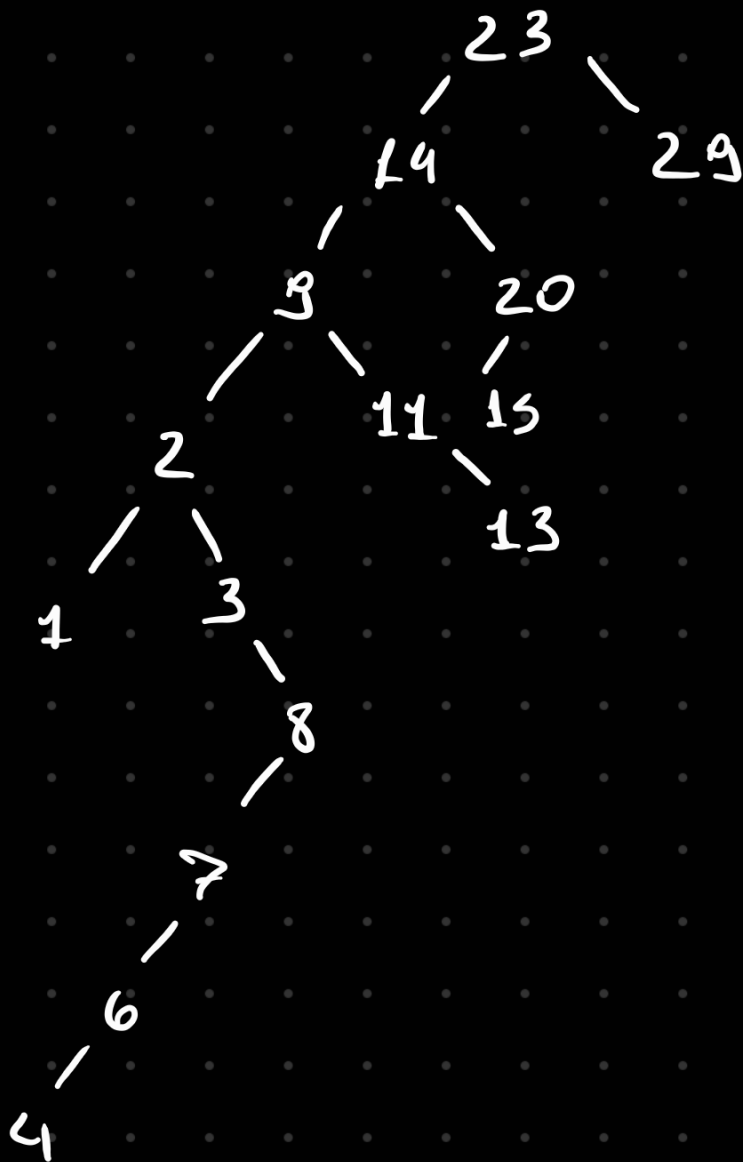


② - Pre-ordem: [30, 30, 10, 45, 39, 53, 90]

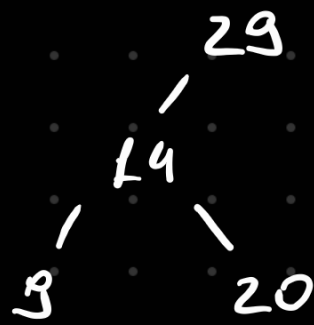
Central: [10, 30, 45, 50, 53, 59, 90]

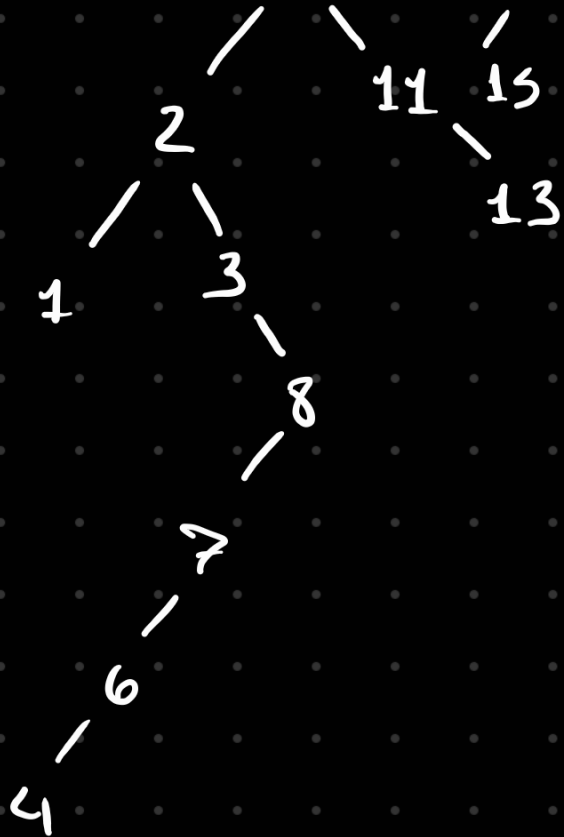
Pós-ordem: [10, 45, 30, 53, 90, 59, 50]

① -

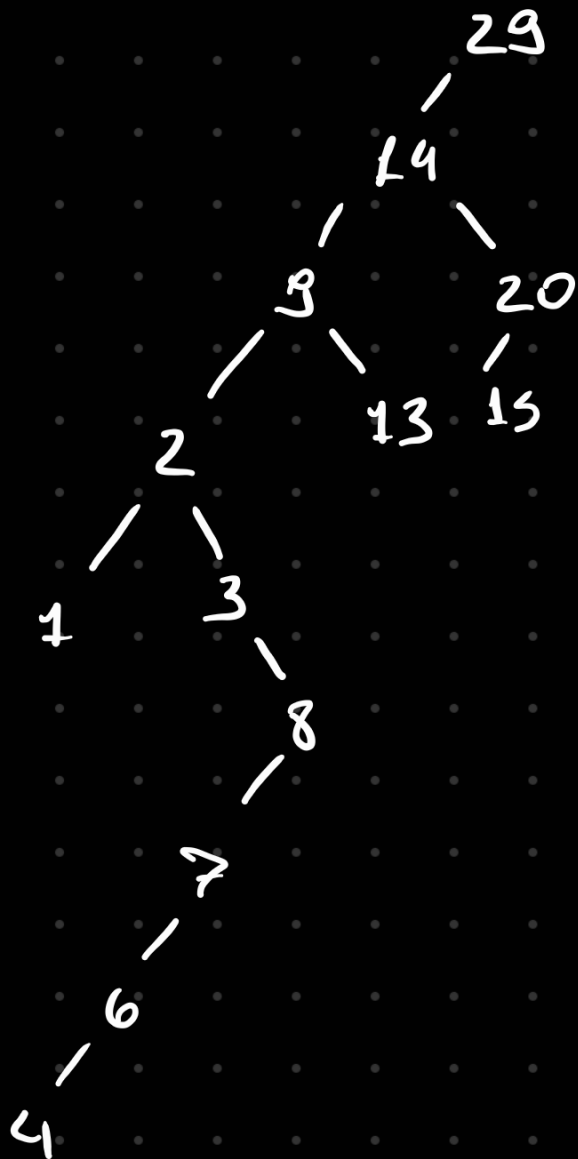


Remover 23:





Remove 11:



Remover 13, 1 e 15

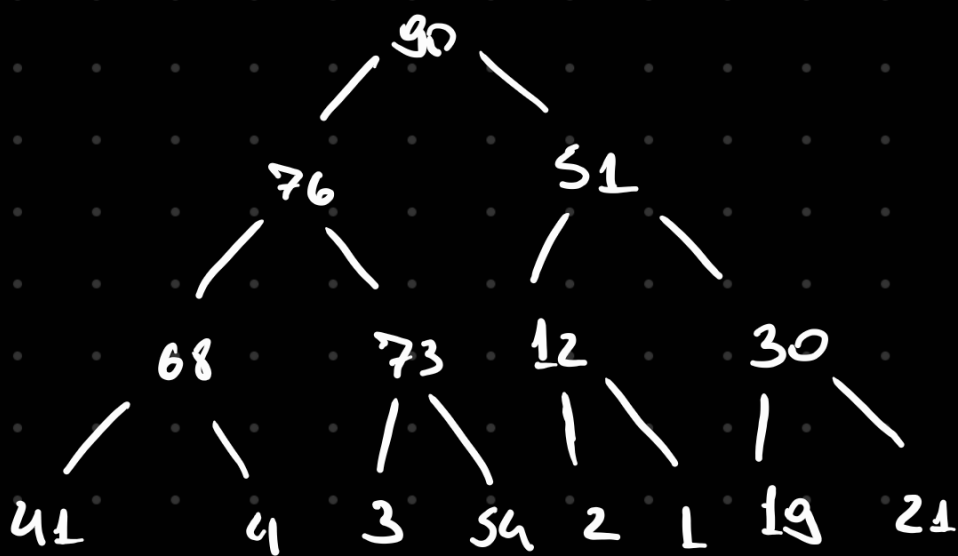
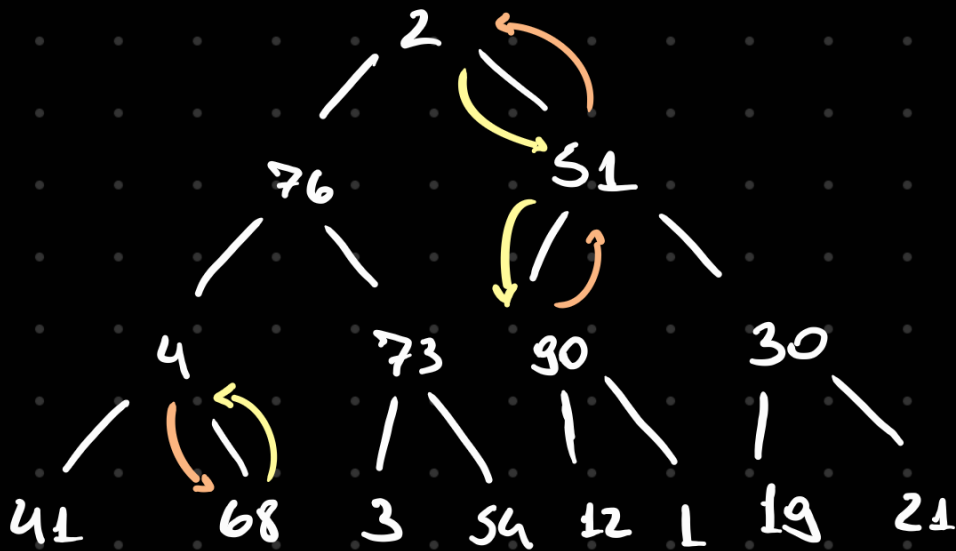


Nesse caso a busca pelo elemento 4 se torna praticamente $O(n)$, pois estamos percorrendo quase todos os elementos da árvore.

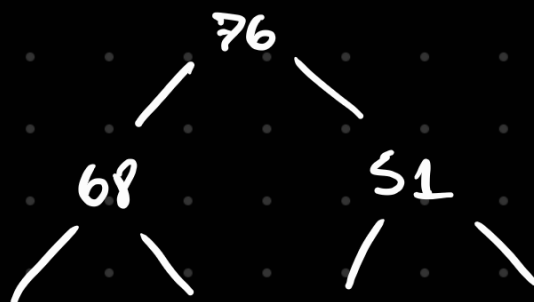
A principal vantagem de ABP é a busca $O(\log n)$, o que só ocorre se a árvore estiver balanceada.

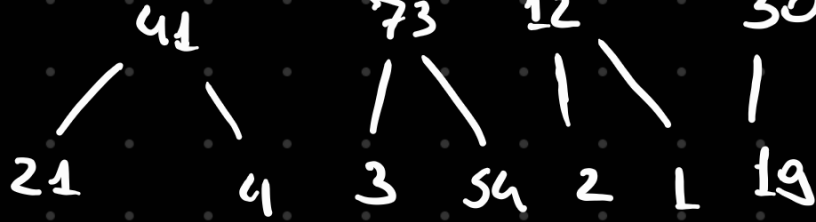
④ - [2, 4, 51, 76, 3, 12, 24, 44, 68, 73, 59, 90, 1, 19, 30]



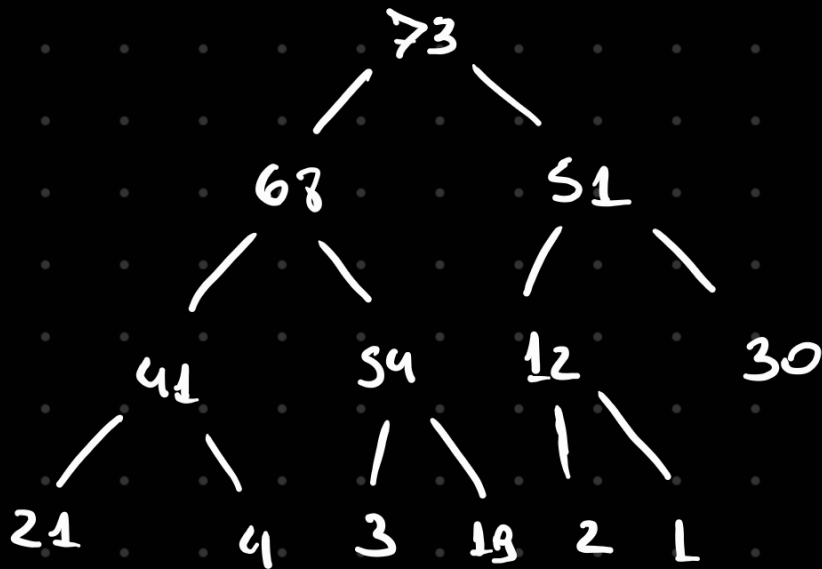


Remover 90

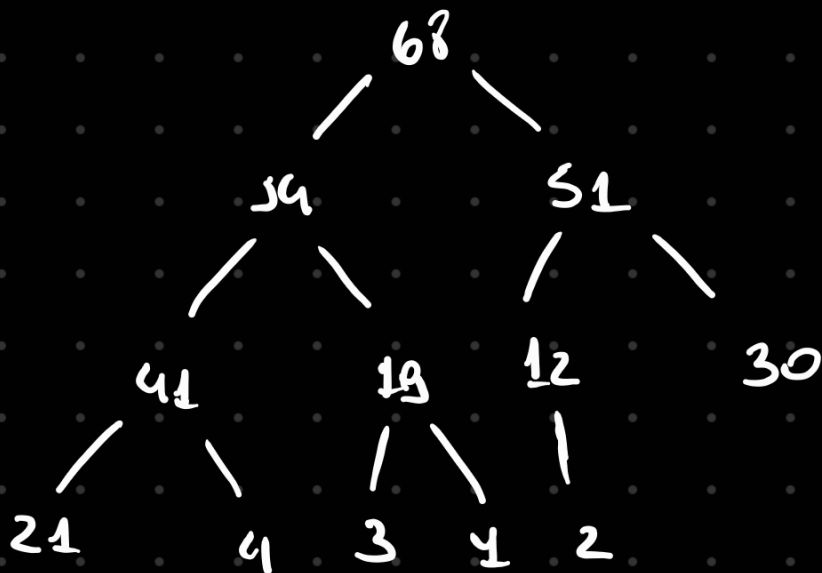




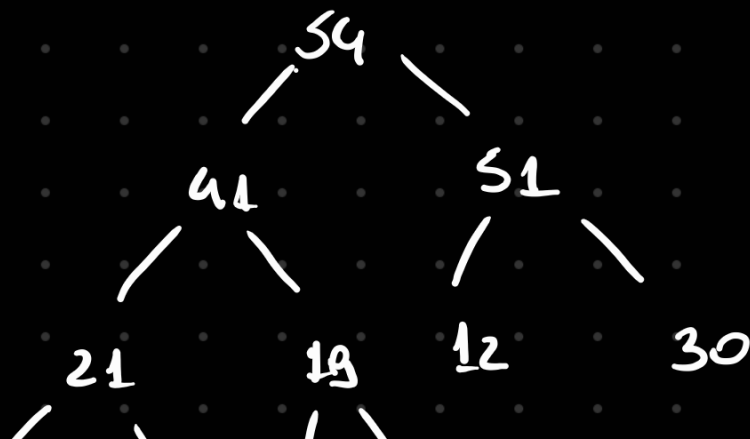
Remover 76:



Remover 73:

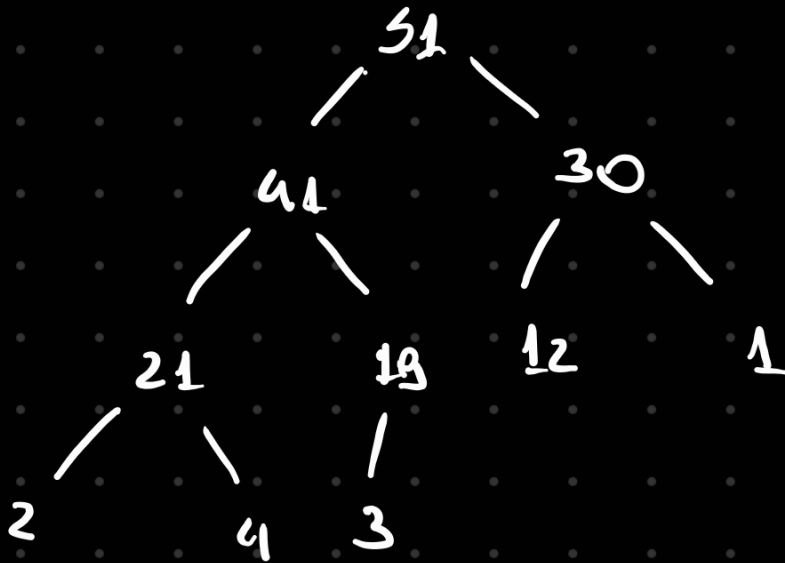


Remover 68:



2 4 3 4

Remover 54:



⑤ -

Letra	Índice
I	9
M	3
P	6
R	8
O	5
F	6
U	1
N	4
P	4
A	1
V	2
E	5
L	2

Com listas:

Divisão

1: U $\rightarrow$ A
2: V $\rightarrow$ L
3: M
4: N $\rightarrow$ D
5: O $\rightarrow$ E
6: P $\rightarrow$ F
7: vazio
8: R
9: I

Endereçamento aberto:

I: 9

M: 3

P: 6

R: 8

O: 5

F: colisão com 6, entra no 7 (se livre)

U: 1

N: 4

D: entra no 0, os outros estão ocupados

A: 2

V: não há mais espaço livre

Houve overflow da tabela hash